Binary Intrusion Detection on the CICIoT2023 Dataset Using a Deep Neural Network

Willis Martin
Department of Cybersecurity
Rochester Institute of Technology
Rochester, NY, USA
wrm3207@rit.edu

***Abstract*—This project implements a deep neural network for binary intrusion detection using a subset of the CICIoT2023 IoT network traffic dataset, classifying traffic as either benign or malicious. We assembled a dataset of 1,535,085 flows from pre-tagged CSV files across CICIoT2023 attack classes. We accounted for class imbalance using capped sampling. We tuned the model's architecture, learning rate, and loss-weighting strategy through repeated runs. The final model reached 88.8% accuracy and an F1-score of 0.923. However, this falls short of the 98-99% accuracy reported in the CICIoT2023 benchmarks.**

***Index Terms*—intrusion detection, deep neural network, IoT security, CICIoT2023**

I. Introduction

As more IoT devices become part of daily life, they present a growing attack surface. As a result, IoT network traffic needs fast, reliable detection. This project trained and evaluated a DNN classifier on the CICIoT2023 dataset [1] to classify network flows as either benign or malicious. This project followed standard practice for train/test separation, leakage-free scaling, and class-imbalance handling. We compare results with the paper's reported binary classification benchmarks.

II. Literature Review

CICIoT2023 [1] is a labeled IoT network traffic dataset covering 33 attacks across seven categories, collected from real IoT devices in a synthetic environment. Each flow has 47 features extracted via CICFlowMeter, including protocol information, packet timing, and statistical flow characteristics. The paper reports reaching accuracy and F1-scores generally above 98% in binary classification across several tested models, using the dataset's full feature set and full row count.

A subsequent survey by Salman and Yong [2] reviewed the CICIoT2023 dataset's structure and applications, noting its adoption as a baseline for binary and multi-class classification studies. They pointed out the dataset's size as a strength for realistic evaluation but a choke point for reproducibility with limited resources.

III. Implementation

A. Dataset

I created a binary-classification subset using the CICIoT2023's per-category CSV files [1]. I retrieved the exact files from the Canadian Institute for Cybersecurity. Rather than using the full 40m row dataset, only one file was selected per attack category to cover all seven of the paper's attack classes: DDoS, DoS, Reconnaissance, Web-based, Brute Force, Spoofing, and Mirai, combined with the benign traffic. This approach keeps the dataset reduced whilst preserving representation covering all major attack categories. We capped each category to stop any single high-volume category from flooding the dataset.

The paper's dataset had a 40:1 malicious-to-benign imbalance. To reduce this to a manageable ratio, the malicious CSV file was capped at 150,000 randomly selected rows using a set seed of 42. For the few cases where a category had fewer than 150,000 rows, we used all rows. After selection, we dropped all rows that contained infinite or NaN values; these typically came from ratio-related features with near-zero duration. In the end, the final dataset contained 1,535,085 rows, with an approximate 1:3.2 ratio. However, I couldn't find a reason for this: the downloaded files contained only 39 numeric features, while the paper reported 47; I used all 39 features as-is.

B. Model

A PyTorch multilayer perceptron with two hidden layers containing 128 and 64 units, respectively, ReLU activations, and dropout p = 0.1. The two-unit output layer was trained with Adam, using a learning rate of 0.001 and a weight decay of 0.0001, with a batch size of 64. During development, we evaluated this configuration over both 10 and 30 training epochs; the longer 30-epoch runs produced results comparable to the 10-epoch runs without a meaningful gain. Because the automated grading harness enforces a 15-minute timeout on the CPU-based reproduction command, we scaled the final training run back to 10 epochs so the project remains reliably reproducible within that constraint. Features were standardized using training-split statistics only, with an 80/20 train/test split.

C. Hyperparameter Tuning

We compared architecture size across four variants ranging from 64 to 256 hidden units in two to three layers, learning rates of 0.001 versus 0.003, and unweighted or class-weighted loss over a ten-run sweep, each trained for 10 epochs, shown in Table I. The top performer used unweighted loss, the 128 and 64 architecture, and a learning rate of 0.001; this configuration was adopted as the final model. Comparing all runs shows that larger architectures provided negligible improvement, and all

runs using a learning rate of 0.003 performed worse than those using 0.001. Additionally, class weights underperformed at this architecture and were consistently worse than the unweighted model, reducing recall from 0.918 to 0.832. This is likely because we reduced the dataset imbalance to 1:3.2 compared to the base 1:40 imbalance, removing the need to correct it with inverse-frequency weighting.

TABLE I
HYPERPARAMETER SWEEP RESULTS (10 EPOCHS, UNWEIGHTED LOSS)

Architecture (lr)	Recall	F1	AUC
[128,64] (0.001)	0.918	0.929	0.956
[256,128,64] (0.001)	0.915	0.928	0.956
[256,128] (0.001)	0.910	0.927	0.955
[64,64] (0.001)	0.906	0.927	0.955
[128,64,32] (0.001)	0.897	0.926	0.956
[128,64,32] (0.003)	0.876	0.918	0.950

The weighting scheme assigned a weight of 2.12 to the benign class and 0.65 to the malicious class, pushing the model to under-predict the majority class more than the imbalance required. This suggests that class weighting, while typically used for severe imbalance, can be counterproductive on datasets with more moderate imbalance and is worth testing on a case-by-case basis rather than applying by default.

TABLE II
EFFECT OF CLASS WEIGHTING ([128,64], 10 EPOCHS)

Loss	Precision	Recall	F1
Weighted	0.992	0.832	0.905
Unweighted	0.945	0.910	0.927

IV. RESULTS AND DISCUSSION

Final performance on the held-out test set, using the selected configuration of hidden sizes 128 and 64, unweighted loss, and 10 training epochs, is summarized in Table III.

TABLE III
FINAL MODEL PERFORMANCE

Metric	Value
Accuracy	0.8879
Precision	0.9709
Recall	0.8796
F1-Score	0.9230
ROC-AUC	0.9555

The confusion matrix (Fig. 1) shows the distribution of correct and incorrect predictions across both classes.

The model shows a more pronounced precision-recall trade-off than earlier tested configurations: precision is 0.971 while recall is 0.880. The model rarely misclassifies benign traffic as malicious, but it misses roughly 12% of the actual malicious flows. For intrusion detection, this trade-off matters, since missing malicious flows is generally worse than falsely flagging benign traffic.

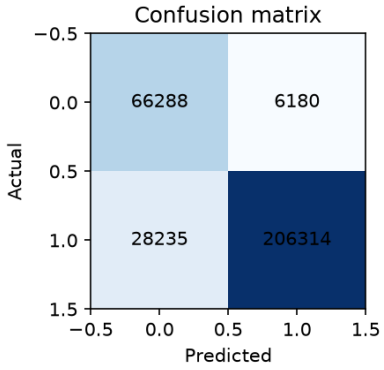


Fig. 1. Confusion matrix on the held-out test set.

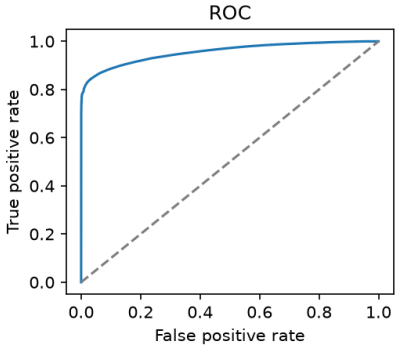


Fig. 2. ROC curve on the held-out test set.

The CICIoT2023 paper reported binary classification of approximately 98-99% across accuracy, precision, recall, and F1. Multiple reasons explain why our model performed notably worse than the one in the paper. One major reason is the feature set: our dataset had access to only 39 features compared to the 47 features reported, meaning we are working with nearly 20% fewer features. Second, for performance reasons, we trained our model on a capped, subsampled dataset rather than the full dataset. Lastly, our model used a simple two-layer architecture, while the top performers in the study likely used significantly larger models.

A. Limitations

This study suffered from multiple limitations. First, the initial data available contained only 39 features, compared with the initial studies' 47. Next, for performance reasons, the training dataset was reduced substantially. Additionally, the final training run was limited to 10 epochs rather than the longer 30-epoch runs tested during development, since the automated grading harness times out any command running longer than 15 minutes on CPU, and 30-epoch training exceeded that window. Also, the dropout value of 0.1 and weight decay value of 0.0001 were not altered in the sweep. This sweep used a single train/test split instead of cross-validation;

as a result, these metrics reflect one random sample of the larger dataset rather than multiple samples.

V. Conclusion

Our two-layer DNN achieved a 0.923 F1 score using a subsampled CICIoT2023 binary classification task. While this is below the paper's 98-99% benchmark, this is attributable to a reduced feature set and dataset scope. Systematic tuning confirmed that unweighted loss and moderate architecture sizes of 128 and 64 were optimal among all tested configurations. Additionally, class-weighted loss and larger architectures both produced worse results than their alternatives. The key lesson I took away was that class-weighting, while typically recommended for imbalanced classifications, is not universally beneficial. Empirical testing against unweighted baselines was required rather than applying weighting by default. Another observation was that repeated training runs with the same fixed seed and configuration produced small variations in reported metrics across separate executions, particularly on CPU. This is consistent with minor nondeterminism in multi-threaded execution rather than a flaw in the training procedure, and it underscores that a single run's metrics should be interpreted with some tolerance rather than treated as an exact reproducible value. Future work includes comparing our model to a model that incorporates the full feature set, uses the entire dataset, and testing results using 8-class and 34-class classification tasks benchmarked in the cited paper.

Appendix

Code repository: https://github.com/ZeroZer-0/csec-520-620-p1-wrm3207. Reproducible via make reproduce (see repository README).

References

[1] Neto ECP, Dadkhah S, Ferreira R, Zohourian A, Lu R, Ghorbani AA. CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment. Sensors. 2023;23(13). Available from: https://www.mdpi.com/1424-8220/23/13/5941.

[2] Salman WAH, Yong CH. Overview of the CICIoT2023 dataset for Internet of Things intrusion detection systems. Mesopotamian Journal of Big Data. 2025;5(1):50-60.